VISIBLE AND CONTROLLABLE MODULES FOR HUMAN OVERSIGHT OF CODING AGENTS

Alexia Huang & Qi Wu
Stanford University

ABSTRACT

Coding agents perform many connected actions that can be difficult for developers to follow and control. We propose organizing execution into visible, controllable modules that connect plans, actions, and code changes. We will compare this interface with a conventional execution log to evaluate human understanding, error detection, and intervention.

1 PROBLEM DEFINITION AND MOTIVATION

The core problem we will address is that developers have difficulty understanding and controlling what a coding agent is doing during a complex task. For example, an agent adding a feature may explore files, change several components, and run tests. As these actions accumulate, developers may lose track of the agent's current goal, the reasons for its changes, and where to intervene. An early mistake, such as misunderstanding an API, may also affect later implementation steps.

This problem is meaningful for developers to better monitor agent behavior, because they need to assess progress, guide decisions, and correct mistakes before those mistakes spread. Better visibility and control are useful throughout execution, even when the agent has not made an obvious error.

Our proposed solution is to organize a single-session agent execution into visible, controllable modules, which represent task-specific subgoals; each may include exploration, edits, tests, and repairs. Each module exposes its purpose, actions, inputs, outputs, and connections to other modules. Developers can inspect the work, pause execution, change instructions, and request revisions at meaningful boundaries.

An agentic system is a good fit for the underlying coding tasks because implementing a feature requires exploring unfamiliar code, choosing tools, and revising the approach as new information or test failures emerge. A fixed script cannot easily prescribe all these decisions. Our interface supports this adaptability while making the agent's changing execution easier for humans to follow and control.

The project fits CS329Z through its focus on tool use, agent architecture, evaluation, and human oversight. Human inspection and control are our primary goals. If time permits, we will investigate cheaper, more reliable agent self-correction, followed by module reuse for similar tasks. Coding implementation is our main use case, although the approach may also apply to other complex tasks.

2 COMPARISON WITH EXISTING WORK

Existing systems and papers address several parts of this problem. ReAct interleaves reasoning and actions to support adaptive task solving (Yao et al., 2023). LangGraph supports explicit workflow state, interruption, and resumption (LangChain, n.d.a), while LangSmith provides execution tracing and monitoring, including model and tool calls within individual agent sessions (LangChain, n.d.b). AgentLens provides hierarchical views of autonomous-agent events and evaluates how users investigate agent behavior (Lu et al., 2024). Agent Skills package dynamically loaded instructions and supporting resources (Zhang et al., 2025).

The core gap we will investigate is how to make a single agent's ongoing behavior visible and controllable. Our goal is to organize the work performed by one agent into meaningful modules that

developers can inspect and guide while the task progresses. These modules represent parts of the same agent's execution, rather than separate agents or independently configured model calls.

Our project builds on existing tracing, visualization, and interruption mechanisms by connecting each module to the agent's instructions, actions, code changes, and test results. Developers can see what the agent is trying to accomplish, pause at module boundaries, and change instructions to guide its subsequent behavior. The agent can revise its plan and module structure as new information emerges.

Our proposed contribution is the design and evaluation of this modular interface within a single coding-agent session. We will investigate whether it improves human understanding, error localization, and intervention compared with a conventional chronological execution log containing equivalent information and controls.

3 Next Steps and Direction

3.1 Proposed System and Scope

We will build a modular inspection and control interface around a single coding agent operating on small repositories. The agent will inspect files, edit code, and run tests while proposing task-specific modules and explicitly announcing module transitions. The extension will associate tool calls and results with the active module and stream execution records to the interface. As the agent learns about the codebase, it may revise its plan while preserving earlier module definitions and execution records. The initial prototype will support one active module at a time and retain the history of revisions and repeated execution attempts. The system will contain four components:

Module representation. Modules have stable identifiers, instructions or skill references, inputs, outputs, dependencies, and validation criteria. Declared prerequisites are distinguished from recorded file or artifact dependencies; inferred links are labeled. Each module has a stable identity, versioned definitions, and separately recorded execution attempts; the graph shows the current module structure, while the timeline preserves actual execution history.

Execution capture. An adapter records tool calls, results, code diffs, test outcomes, and module assignments. Unassigned actions remain visible. Raw runtime events are checked against captured records to identify omissions.

Inspection interface. A graph shows modules and their status, with a linked timeline of actual execution. Selecting a module reveals its purpose, instructions, actions, code changes, and checks. Users can examine differences between the plan and execution.

Intervention controller. Users can pause at module boundaries, edit instructions, and request a revision before resuming. Edits create new module versions. At each recoverable module boundary, we will associate the agent's conversation state and module-plan version with a workspace snapshot. Recovery will restore these together, invalidate subsequent outputs, and conservatively rerun subsequent work.

3.2 Evaluation

We will determine whether the approach works by comparing the modular interface with a conventional execution log that provides equivalent information and controls. Our primary success criteria are better human understanding of the agent's work, more accurate error localization, and more successful intervention, without a meaningful increase in false alarms. We will evaluate these criteria through a trace-inspection study and an interactive intervention study.

Tasks and grading. We will curate small feature additions and bug fixes in public repositories, recording task sources, repository versions, and selection criteria. Each task will have human-authored requirements, behavioral tests, and checks for regressions. Tests and blinded human review will assess correctness. Pilot tasks will be separate from evaluation tasks. Traces will include clean runs, natural failures, and reviewed seeded failures, such as incorrect API assumptions, omitted requirements, and misinterpreted tests. Seeded errors will have annotated origins and affected changes; natural and seeded failures will be analyzed separately.

Trace-inspection study. We will render the same execution records in either the modular interface or a searchable chronological log, exposing equivalent plans, events, diffs, and test results. Comprehension questions will assess whether users identify the agent's current goal, completed changes, and downstream dependencies. Answers will be graded against a human-reviewed rubric. We will also measure error detection, localization, false alarms on clean runs, and inspection time.

Interactive intervention study. Both interfaces will provide equivalent pause, instruction-editing, and revision controls, with matched models, tools, and budgets. Trials will begin from matched execution states and use a fixed intervention time budget. We will measure intervention time, unnecessary interventions, and correction success. Successful correction requires resolving the target error while preserving other task requirements and avoiding regressions. Final implementation quality will be assessed with tests and blinded review.

Study design and analysis. We will pilot with 3–4 developers and provisionally target 12–16 participants familiar with coding agents, refining the study after assessing task duration and recruitment. A within-participant design will counterbalance interfaces and use distinct matched tasks. We will report paired effects and uncertainty intervals, accounting for repeated observations by participant and task. Scoring rubrics and analysis procedures will be specified before the main study. Inconclusive findings will be reported explicitly.

3.3 Milestones and Optional Extensions

Our next steps are to establish module assignment, execution capture, and rendering during weeks 3–5. During weeks 6–7, we will add boundary interventions and conduct a pilot for the midpoint demo. The remaining weeks will focus on comparative evaluation and failure analysis. Early integration checks will test workspace restoration and dependency handling; early recruitment will address study feasibility.

If time permits, our first extension will investigate cheaper, more reliable agent self-correction. We will compare agent-driven module correction with ordinary feedback-based correction and full restarts under matched budgets, reporting task success, tokens, tool calls, and recovery time.

If additional time permits, our second extension will investigate module reuse for similar tasks. We will evaluate reuse of validated module procedures on held-out similar coding tasks, measuring construction effort, cost, and correctness. Core success does not depend on either extension.

REFERENCES

LangChain. Interrupts. LangGraph documentation, n.d.a. URL https://docs.langchain.com/oss/python/langgraph/interrupts.

LangChain. LangSmith observability. LangSmith documentation, n.d.b. URL https://docs.langchain.com/langsmith/observability.

Jiaying Lu, Bo Pan, Jieyi Chen, Yingchaojie Feng, Jingyuan Hu, Yuchen Peng, and Wei Chen. AgentLens: Visual analysis for agent behaviors in LLM-based autonomous systems. *arXiv preprint arXiv:2402.08995*, 2024. URL https://arxiv.org/abs/2402.08995.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.03629.

Barry Zhang, Keith Lazuka, and Mahesh Murag. Equipping agents for the real world with Agent Skills. Anthropic Engineering, 2025. URL https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills. Published October 16, 2025.

A AI-USE DISCLOSURE STATEMENT

The project idea, three-stage goals, proposed methods, and evaluation plan were developed by the authors through team brainstorming and discussions with teaching assistants during office hours, without AI assistance. OpenAI Codex was used only to support proposal writing, including refining language, organizing the presentation of the authors' ideas, and preparing LaTeX in the required ICLR format. The authors are responsible for reviewing the proposal and verifying its references and claims before submission.